

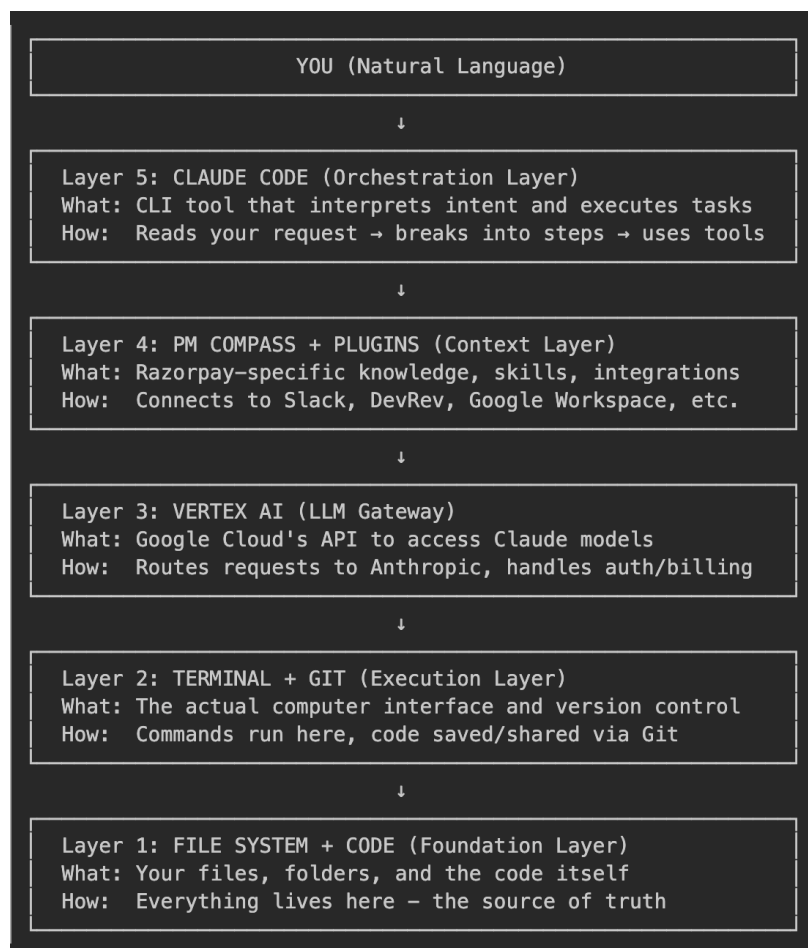
Razorpay AI Dev Stack : Basic 101 and Resources

Purpose: Foundation training for AI-assisted development

Audience: PMs, Designers, non-engineers pushing code for the first time

Approach: Layer-by-layer understanding of the stack

The Stack (Mental Model)



Learning Path

Module 1: Foundation Layer (Files & Code)

Time: 15 min | Prerequisite: None

What You Need to Know

- Code is just text files with specific extensions (.js, .py, .md)
- The **working directory** is "where you are" in the file system

Key Concepts

Concept	What It Means	Example
Path	Address of a file/folder	/Users/you/Desktop/project
Absolute path	Full address from root	/Users/you/code/app.js
Relative path	Address from current location	./src/app.js
Extension	File type indicator	.js = JavaScript, .md = Markdown

Helpful Resources

-  Absolute and Relative Paths (2m 15s)

Module 2: Execution Layer (Terminal)

Time: 30 min | Prerequisite: Module 1

What Is a Terminal?

Terminal is a text-based way to control your computer. Instead of clicking, you type commands.

Why it matters: Claude Code runs IN the terminal. You need basic fluency.

Essential Commands

Command	What It Does	Example
<code>pwd</code>	Print current directory	<code>pwd</code> → <code>/Users/you/Desktop</code>
<code>ls</code>	List files in directory	<code>ls</code> → shows files
<code>ls -la</code>	List ALL files (including hidden)	Shows <code>.git</code> , <code>.env</code> , etc.
<code>cd</code>	Change directory	<code>cd ~/Desktop</code>
<code>cd ..</code>	Go up one level	From <code>/a/b/c</code> → <code>/a/b</code>
<code>mkdir</code>	Make new directory	<code>mkdir my-project</code>
<code>touch</code>	Create empty file	<code>touch app.js</code>
<code>cat</code>	Display file contents	<code>cat readme.md</code>
<code>rm</code>	Delete file	<code>rm test.txt</code>
<code>rm -rf</code>	Delete folder and contents	<code>rm -rf old-folder</code> (careful!)
<code>clear</code>	Clear terminal screen	Tidies up your view

Shortcuts

Shortcut	What It Does
<code>Tab</code>	Autocomplete file/folder names
<code>↑ / ↓</code>	Cycle through command history
<code>Ctrl + C</code>	Cancel current command
<code>Ctrl + L</code>	Clear screen (same as <code>clear</code>)

Helpful Resources

- [Learn Enough Command Line to Be Dangerous](#) (< 1h)
- [Command line crash course | MDN](#) (~ 1h)
- [Absolute BEGINNER Guide to the Mac OS Terminal](#) (17m)

Practice Exercise

None

Open Terminal (Mac: Cmd+Space → "Terminal")

Run these commands in order (one by one):

```
cd ~/Desktop
```

```
mkdir bd2-practice
```

```
cd bd2-practice
```

```
pwd          # Should show: /Users/you/Desktop/bd2-practice
```

```
touch hello.js
```

```
ls           # Should show: hello.js
```

```
echo "console.log('hi')" > hello.js
```

```
cat hello.js  # Should show: console.log('hi')
```

Module 3: Execution Layer (Git)

Time: 45 min | Prerequisite: Module 2

What Is Git?

Git is **version control** - it tracks changes to your code over time.

Key mental model: Think of Git as "save points" in a video game. You can always go back.

Core Concepts

Concept	What It Means	Analogy
Repository (repo)	A project tracked by Git	A book with full revision history
Commit	A saved snapshot	A save point in a game

Branch	A parallel version of code	A fork in the road
main/master	The primary branch	The "real" version
Remote	The server copy (GitHub)	The cloud backup
Clone	Download a repo	Copy a book from library
Push	Upload your changes	Submit your work
Pull	Download latest changes	Get team's updates

The Git Workflow

None

1. PULL latest changes → `git pull origin main`
2. CREATE a branch → `git checkout -b feature-name`
3. MAKE changes → (edit files)
4. STAGE changes → `git add filename.js`
5. COMMIT changes → `git commit -m "message"`
6. PUSH to remote → `git push origin feature-name`
7. CREATE Pull Request → (on GitHub)

Essential Commands

Command	What It Does	When to Use
<code>git status</code>	Show what's changed	Before every commit
<code>git add file.js</code>	Stage specific file	When ready to save
<code>git add .</code>	Stage all changes	When everything is ready
<code>git commit -m "msg"</code>	Save snapshot	After staging

<code>git push</code>	Upload to GitHub	After committing
<code>git pull</code>	Download latest	Start of each session
<code>git checkout -b name</code>	Create new branch	Starting new work
<code>git checkout main</code>	Switch to main branch	Before pulling updates
<code>git log --oneline</code>	Show commit history	To see what's happened
<code>git diff</code>	Show unstaged changes	To review before staging

Helpful Resources

- [📺 Git Explained in 100 Seconds](#) (2m)
- [git - the simple guide - no deep shit!](#) (5m)
- [📺 Git and GitHub for Beginners - Crash Course](#) (1h 8m)
- [What is version control | Atlassian Git Tutorial](#) (~1.5 h) - Includes setup steps

Practice Exercise

None

```
cd ~/Desktop/bd2-practice
```

```
git init      # Initialize Git
```

```
git status    # Check status
```

```
echo "# My First Project" > README.md    # Create and stage a file
```

```
git add README.md
```

```
git commit -m "Initial commit: add README" # Commit
```

```
git log --oneline    # Check log
```

Common Mistakes & Fixes

Mistake	Fix
Committed to wrong branch	<code>git checkout correct-branch</code> then <code>git cherry-pick <commit-hash></code>
Want to undo last commit	<code>git reset --soft HEAD~1</code> (keeps changes)
Forgot to pull before pushing	<code>git pull --rebase origin main</code>
Merge conflicts	Open file, resolve manually, then <code>git add</code> and <code>git commit</code>

Module 4: LLM Gateway (Vertex AI)

Time: 15 min | **Prerequisite:** None (conceptual)

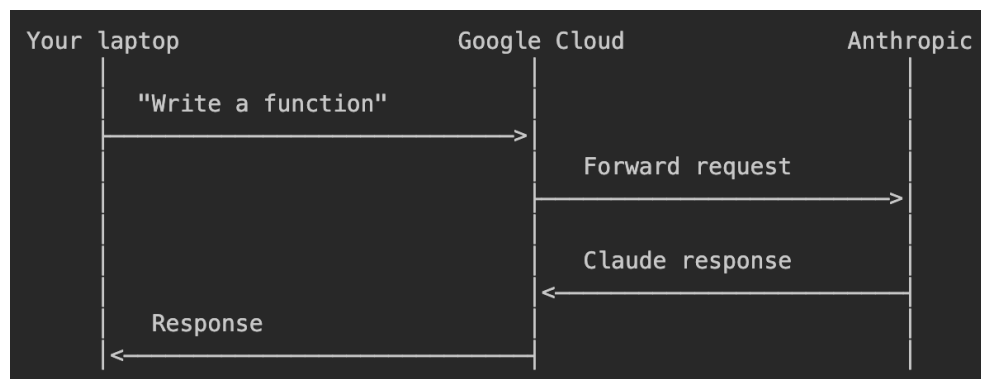
What Is Vertex AI?

Vertex AI is **Google Cloud's gateway** to AI models, including Claude.

Why we use it:

- Razorpay has an enterprise agreement with Google Cloud
- Billing, security, and access control handled centrally
- No personal API keys needed

How It Works



What You Configured

Setting	Value	What It Does
CLAUDE_CODE_USE_VERTEX=1	Enabled	Routes through Vertex, not direct API
ANTHROPIC_VERTEX_PROJECT_ID	product-design-claude-code	Bills to Razorpay's GCP project
CLOUD_ML_REGION	global	Uses nearest available region
gcloud auth	Your Razorpay account	Authenticates you to Google Cloud

You Don't Need to Know

- How Vertex pricing works (handled by org)
- How to configure quotas (handled by admins)
- Model selection (Claude Code picks the right model)

If you're Interested

- [Vertex AI Documentation](#)

Module 5: Context Layer (PM Compass + Plugins)

Time: 20 min | **Prerequisite:** Claude Code installed

What is PM Compass?

PM Compass is **Razorpay-specific knowledge and skills** added to Claude Code.

What it provides:

- Your PM identity and context (from CLAUDE.md)
- Skills like /drafting-strategy, /reviewing-strategy
- Templates for Razorpay-specific documents

What Are Plugins (MCP)?

MCP (Model Context Protocol) servers connect Claude Code to **external tools**.

None

Claude Code

└─ plugin: google-workspace → Reads Google Docs, Sheets, Gmail

└─ plugin: slack-mcp → Searches Slack messages

└─ plugin: devrev → Fetches shipped features

└─ plugin: blade-mcp → Gets Razorpay design system docs

How Plugins Work

When you ask Claude Code something like:

"What did we ship last quarter?"

Claude Code:

1. Recognizes this needs DevRev data
2. Calls the DevRev plugin
3. Plugin queries DevRev API with your auth
4. Returns structured data
5. Claude Code summarizes for you

Available Plugins at Razorpay (Evolving)

Plugin	What It Does	Example Query
Google Workspace	Docs, Sheets, Drive, Calendar	"Find my strategy doc"
Slack	Search messages, channels	"What did Khilan say about X?"
DevRev	Tickets, issues, shipped features	"Show my team's open issues"
Blade	Razorpay design system	"How do I use a Button?"

Helpful Resources

- [Model Context Protocol](#) (2m)
- [What is MCP \(Youtube\)](#) (2m)
- [Connect Claude Code to tools via MCP](#)

Checking Plugin Status

None

In Claude Code, run:

/mcp

This shows which plugins are connected and working.

Module 6: Orchestration Layer (Claude Code)

Time: 30 min | **Prerequisite:** All previous modules

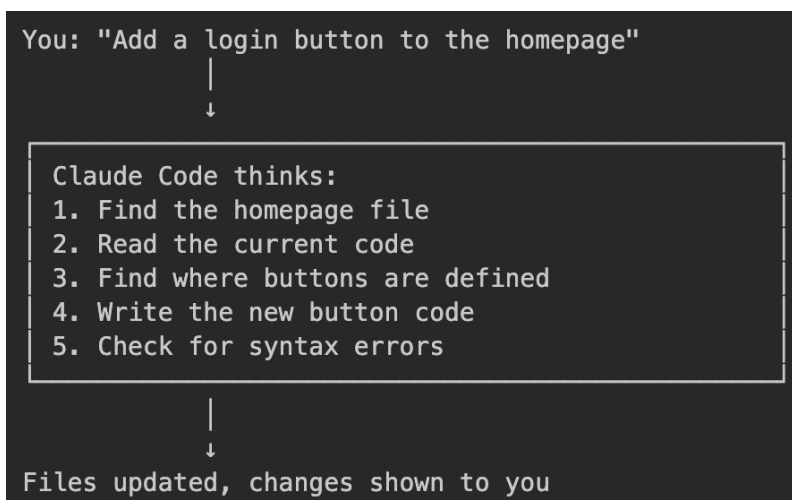
What is Claude Code?

Claude Code is a **CLI tool** that interprets your natural language and executes development tasks.

Mental model: It's an AI agent that can:

- Read and write files
- Run terminal commands
- Search codebases
- Call external tools (via plugins)
- Chain complex operations

How It Works



Key Commands

Command	What It Does
<code>claude</code>	Start Claude Code in current directory
<code>/help</code>	Show all available commands
<code>/mcp</code>	Check plugin status
<code>/clear</code>	Clear conversation history
<code>/exit</code>	Exit Claude Code

Working with Claude Code

Good prompts:

- "Read the README and summarize what this project does"
- "Find all files that handle user authentication"
- "Add error handling to the submitForm function in src/form.js"
- "Run the tests and fix any failures"

Bad prompts:

- "Make it better" (too vague)
- "Build me an app" (too broad)
- "Do whatever" (no direction)

The Permission System

Claude Code asks permission before:

- Writing/editing files
- Running commands
- Accessing external services

Options when prompted:

- `y` = Yes, do it
- `n` = No, don't
- `a` = Yes, and don't ask again for this session

Helpful Resources

- [Claude Code Tutorial - YouTube](#) (Total playlist < 1.5h)
- [Common workflows - Claude Code Docs](#)

- Just ask it directly!

Practice Exercise

None

```
cd ~/Desktop/bd2-practice
```

```
claude
```

Try these prompts:

1. "What files are in this directory?"

2. "Create a simple HTML file with a hello world heading"

3. "Add a button that shows an alert when clicked"

4. `/exit`

Quick Reference Card

Terminal Essentials

None

```
pwd      # Where am I?
```

```
ls        # What's here?
```

```
cd <folder>  # Go to folder
```

```
cd ..      # Go up
```

```
mkdir name  # Make folder
```

```
touch file.js  # Make file
```

```
cat file.js  # Show contents
```

Git Essentials

None

```
git status      # What's changed?
git add file    # Stage for commit
git commit -m "msg" # Save snapshot
git push        # Upload to GitHub
git pull        # Download updates
git checkout -b x # New branch named x
```

Claude Code Essentials

None

```
claude      # Start
/help       # Commands
/mcp        # Check plugins
/clear      # Reset conversation
/exit       # Quit
```

Glossary

Term	Definition
CLI	Command Line Interface - text-based way to use software
Repository	A project folder tracked by Git
Commit	A saved snapshot of code changes
Branch	A parallel version of code for isolated work
Push/Pull	Upload/download code to/from GitHub
MCP	Model Context Protocol - how plugins connect to Claude

Vertex AI	Google's gateway to AI models
------------------	-------------------------------

Plugin	External service integration (Slack, DevRev, etc.)
---------------	--
